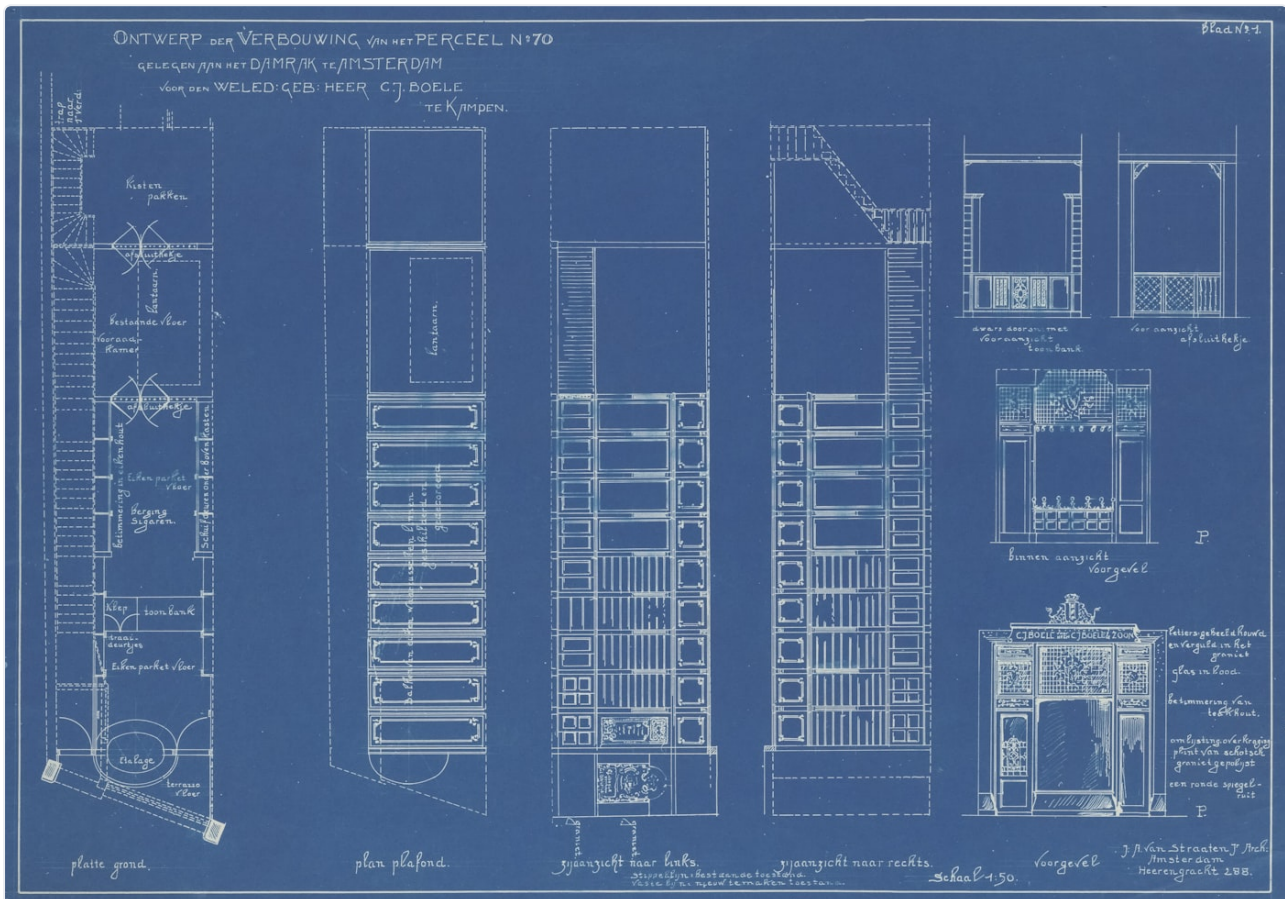


# A Simple Blueprint for Building Software with AI Agents

*Stealing the best of Spec Kit, Superpowers and GStack into one workflow a data scientist can actually follow, from idea to shipped.*

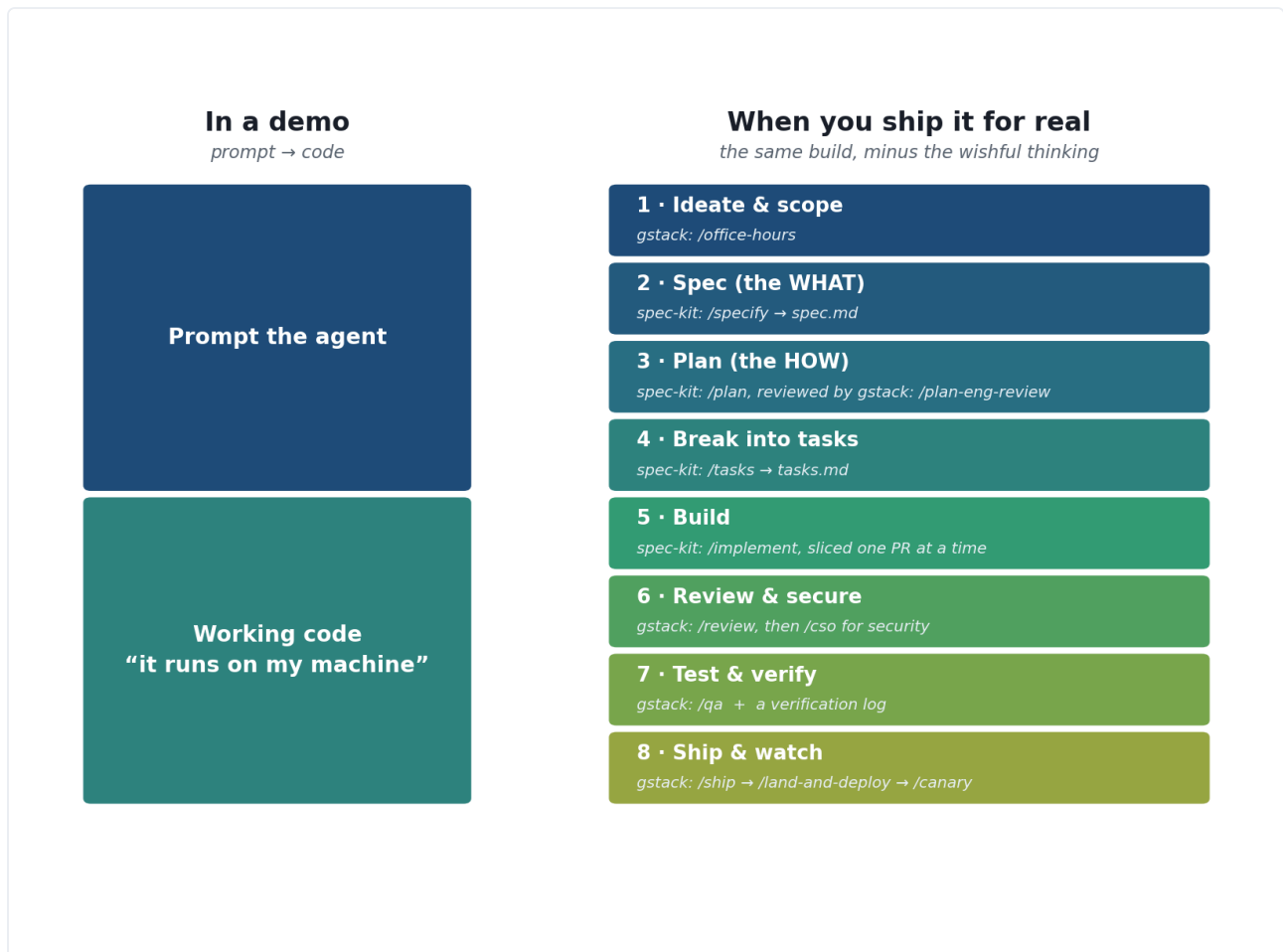


*Photo by Amsterdam City Archives on Unsplash*

I'll be honest about where I'm coming from. I've spent years in data science, and for most of that time my "development process" was a notebook, a lot of cells run out of order, and faith that I'd remember what I did last Tuesday. It works right up until the thing you built has to survive being used by someone who isn't you. Data scientists learn to model. We rarely learn the software development lifecycle around the model: the spec, the plan, the review, the tests, the boring paperwork that stops a Friday deploy becoming a Saturday incident.

Then AI coding agents arrived, and with them a wave of frameworks that bolt a real engineering process onto the agent. Three kept coming up in my setup: **Spec Kit**, **Superpowers**, and **GStack**. Each is good. Between them they also carry something like sixty slash-commands, which is a lot to stare at on a Monday. So I did what I do with any new tool: tried them, got confused, and distilled the confusion into one blueprint I could remember. The

remarks below are my personal take basis usage, not a benchmark, and there's no promotion here.



*Image by Author*

## Vibe coding got us here, and where it stops

Andrej Karpathy coined “vibe coding” in early 2025: you give in to the vibes and, in his words, *“Accept All” always, I don’t read the diffs anymore.*

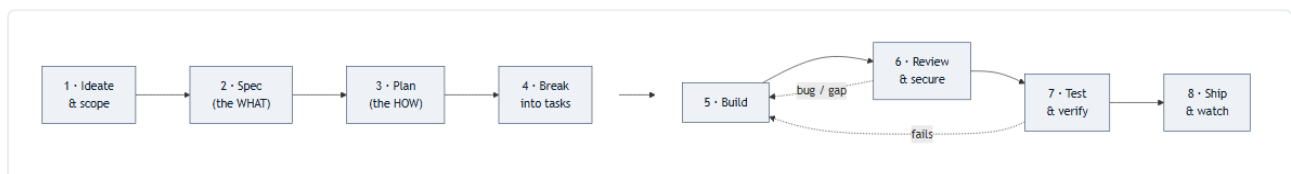
For a weekend throwaway it’s genuinely great. The trouble starts on the last stretch. Addy Osmani calls it the 70% problem: the agent sprints you to 70 percent, then the final 30 becomes “whack-a-mole with code you don’t fully understand.” Simon Willison’s definition lands it, vibe coding is “generating code with AI without caring about the code that is produced.” Fine when it’s just you; risky the moment it touches production or a teammate.

The fix isn’t to stop using agents. It’s to give the agent a *document*

to build against instead of a vibe, so every change is small and reviewable. That is the whole point of the blueprint below.

## A one-minute primer on the three toolkits

- **Spec Kit** (GitHub): the planning engine. It turns an idea into readable Markdown, `spec.md` then `plan.md` then `tasks.md`, so the agent codes against a document.
- **GStack**: the review-and-ship engine. A command for every late stage: `/office-hours`, `/plan-eng-review`, `/review`, `/cso`, `/qa`, `/ship`, `/canary`.
- **Superpowers** (obra): the discipline, real test-driven development and an append-only verification log. It's actually the most-starred of the three (roughly 243k stars to Spec Kit's 117k), but it bakes its method into agent behaviour and runs only on Claude. I still anchor on Spec Kit, because a newcomer learns the method faster when it lives in files you can read, and I borrow just one Superpowers habit later.



*Image by Author*

## The blueprint, stage by stage

Pick the stages your work has actually earned. A throwaway script needs almost none of this; anything another human will touch needs most of it.

- 1. Ideate and scope.** Before any code, ask whether the thing is worth building. GStack's `/office-hours` challenges the premise and offers alternatives instead of rubber-stamping your first idea. The most expensive code is the code you shouldn't have written.
- 2. Spec, the WHAT.** Spec Kit's `/specify` writes `spec.md`: what you're building, success criteria, edge cases, and no implementation detail. `/clarify` then makes you quantify the vague words, so "fast" becomes "under 200ms". This single habit changed my output the most.
- 3. Plan, the HOW.** `/plan` turns the spec into an architecture in `plan.md`. Hand that to GStack's `/plan-eng-review`. Spec Kit writes the plan; GStack asks whether it's any good. Catching a design flaw here, on a Markdown file, is far cheaper than catching it in code.
- 4. Break it into tasks.** `/tasks` splits the plan into `tasks.md`, a list of small, ordered, testable steps. Now both you and the agent always know what "done" means.

**5. Build.** `/implement` works the task list. The one habit I borrow from Superpowers: build in small slices, one pull request each, never a 4,000-line blob nobody can review. The trade-off is real, small PRs are slower to ship and much faster to trust.

**6. Review and secure.** An agent reviewing its own code has the same blind spot as a student grading their own exam. GStack's `/review` is the gate every diff should pass. For anything security-sensitive, add `/cso` for an OWASP-style audit, and `/codex` when a genuinely different model's second opinion is worth the extra step. Security is a named stage here, not a vague good intention.

**7. Test and verify.** `/qa` drives a real browser through every route, finds bugs, and fixes them. Then keep the proof: a short verification log stamped with the commit, the test, and the result. It feels like paperwork until the day someone asks "was this actually tested?" and you answer with a commit hash instead of a shrug.

**8. Ship and watch.** `/ship` opens the PR, `/land-and-deploy` merges and deploys, `/canary` watches production for errors afterwards. The unglamorous tail that data science training skips entirely.

## The blueprint is really a harness

---

Look back at those eight stages and notice what they share: almost every one exists because an agent, left alone, will confidently skip it. The spec stops it inventing scope. Review and security stop it shipping a plausible-looking bug. The verification log stops "it ran on my machine" being the only record. So this isn't really a process for *you*. It's a harness around the *agent*, the rails that turn raw speed into something you'd trust on a Monday morning. As the models get faster, the harness is what keeps that speed safe to use, and that, more than any single command, was the piece I'd been missing.

## Learnings

---

- The process is the point, not the tool. The real unlock was internalising the sequence and the reason for each step; the toolkits just make each step cheap.
- Spec Kit plans, GStack ships. If you remember one split, remember that one.
- Security and verification were my two biggest gaps, and they're exactly the two a modeller's training never covers. Making them named, non-skippable steps did more for me than any amount of better prompting.

If you're a data or ML person who has always felt slightly fraudulent about the "software" half of the job, this is the scaffolding I wish I'd had. Steal it, drop the stages your work hasn't earned, and make it yours.

## References

---

1. Andrej Karpathy, the original "vibe coding" post (Feb 2025).  
<https://x.com/karpathy/status/1886192184808149383>
2. Simon Willison, "Not all AI-assisted programming is vibe coding."  
<https://simonwillison.net/2025/Mar/19/vibe-coding/>
3. Addy Osmani, "The 70% problem: hard truths about AI-assisted coding."  
<https://addyo.substack.com/p/the-70-problem-hard-truths-about>
4. Spec-Driven Development with AI, The GitHub Blog. <https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-toolkit/>
5. Spec Kit, GitHub's spec-driven development toolkit. <https://github.com/github/spec-kit>
6. Superpowers, Jesse Vincent (obra). <https://github.com/obra/superpowers>
7. GStack, the review-and-ship skill collection referenced here, from my own Claude Code setup.